Q8. Debugging String Matching Code

Reg. No: 192511075 | Name: Arun Balaji S J

1. Given (Buggy) Code

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    for i in range(n - m):
        for j in range(m):
            if text[i+j] != pattern[j]:
                break
        if j == m-1:
            print("Match at index", i)
```

2. Logical Errors Identified

Error 1: Incorrect outer loop bound

The outer loop is written as range(n - m). Valid starting positions for an m-length pattern inside an n-length text run from index 0 up to and including index (n - m) — that is (n - m + 1) possible positions in total. Since Python's range(n - m) stops one short (it only produces 0 .. n-m-1), the very last valid starting index, i = n - m, is never checked, so a match that begins at the last possible position is always skipped.

Error 2: Unsafe match check using 'j == m-1' after break

After the inner for loop, the code tests 'if j == m-1' to decide whether a match occurred. This is unreliable because the variable j holds whatever value it had when the inner loop stopped, regardless of whether it stopped due to a completed scan or a mismatch:

- If all m characters match, the loop finishes normally with j = m-1 (its last iteration value) -> correctly detected as a match.
- If a mismatch occurs exactly on the LAST character of the pattern (j = m-1), the 'break' statement fires with j still equal to m-1 -> the code wrongly reports a match even though the last character did not match (a false positive).
- If m = 0 (empty pattern) or the inner loop never runs, j is undefined at the point of the check, which would raise a NameError.

In short, 'j == m-1' cannot distinguish 'the loop completed successfully' from 'the loop broke on its final iteration', so some non-matches are reported as matches, and (combined with Error 1) some genuine matches at the last valid index are skipped entirely.

3. Why Matches Get Skipped -- Summary

- The off-by-one outer range range(n - m) permanently excludes the last valid alignment i = n - m, so any occurrence of the pattern that starts exactly there is never even attempted.
- Even for alignments that ARE checked, the flawed j == m-1 test can report false matches, which undermines correctness of the results that ARE printed.

4. Corrected and Commented Code

```
def string_match(text, pattern):
    """
    Brute-force string matching (corrected).
    Finds and prints every starting index in 'text' where 'pattern' occurs.
    """
    n, m = len(text), len(pattern)

    # Edge case: empty pattern or pattern longer than text
    if m == 0 or m > n:
        return

    # FIX 1: loop must include the LAST valid starting index (n - m),
    # so the range upper bound is (n - m + 1), not (n - m).
    for i in range(n - m + 1):
        j = 0
        # Compare pattern with text[i .. i+m-1] character by character.
        # The comparison stops (redundant work avoided) as soon as a
        # mismatch is found.
        while j < m and text[i + j] == pattern[j]:
            j += 1

        # FIX 2: a match is confirmed only when the inner loop reached
        # the end of the pattern (j == m), NOT when it merely broke
        # while j happened to equal m-1. Using a dedicated counter/
        # while-condition removes the ambiguity of the old 'break' +
        # 'j == m-1' check.
        if j == m:
            print("Match at index", i)
```

5. Optimization -- Avoiding Redundant Comparisons

Within a single alignment, the corrected code already exits the inner while loop the instant a mismatch is found, so it never wastes time comparing the remaining characters of that alignment (this is the standard 'early exit' optimization for brute-force matching).

However, plain brute force still re-examines text characters from scratch at every new alignment i, which is wasteful when the pattern has repeating prefixes. A further, more powerful optimization is the Knuth-Morris-Pratt (KMP) algorithm, which pre-computes a Longest-Prefix-Suffix (LPS) table for the pattern and uses it to skip re-comparison of text characters that are already known to match from the previous attempt. This reduces the number of character comparisons from the worst-case O(n*m) of brute force down to O(n + m):

```
def build_lps(pattern):
    """Build the Longest-Prefix-Suffix table used by KMP."""
    m = len(pattern)
    lps = [0] * m
    length = 0        # length of the previous longest prefix-suffix
    i = 1
    while i < m:
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
```

```
            i += 1
        elif length != 0:
            length = lps[length - 1]   # fall back, no i increment
        else:
            lps[i] = 0
            i += 1
    return lps

def kmp_search(text, pattern):
    """
    KMP string matching -- avoids re-scanning text characters that
    are already known to match, giving O(n + m) time overall.
    """
    n, m = len(text), len(pattern)
    if m == 0 or m > n:
        return

    lps = build_lps(pattern)
    i = j = 0                  # i -> text index, j -> pattern index
    while i < n:
        if text[i] == pattern[j]:
            i += 1
            j += 1
            if j == m:
                print("Match at index", i - j)
                j = lps[j - 1]       # continue searching for overlaps
        else:
            if j != 0:
                j = lps[j - 1]       # use LPS table, do NOT move i back
            else:
                i += 1
```

6. Time Complexity Analysis

Corrected brute-force version

- Best case: O(n) -- e.g. when the first character of the pattern never matches the text, each alignment is rejected in O(1).
- Worst case: O((n - m + 1) * m), i.e. O(n * m) -- e.g. text = "aaaa...a" and pattern = "aaa...ab", where almost every alignment matches m-1 characters before failing.
- Space complexity: O(1) extra space (no auxiliary data structures).

Optimized (KMP) version

- Building the LPS table: O(m).
- Scanning the text: O(n), because the text pointer i never moves backward and the pattern pointer j is bounded by the LPS table.
- Total time complexity: O(n + m), with O(m) extra space for the LPS array.

7. Conclusion

The original code had two logical bugs: an off-by-one error in the outer loop's range that skipped the last valid starting index, and an unsafe post-loop check (j == m-1) that could not reliably tell a completed match from a break on the final character. Fixing the range to n - m + 1 and replacing the check with a while-loop counter that must reach exactly m guarantees every occurrence is found correctly. The early-exit inner loop keeps the brute-force version efficient at O(n * m) worst case, while switching to KMP further improves it to a guaranteed O(n + m).